RELATÓRIO TÉCNICO DE DESENVOLVIMENTO: PROCESSADOR FALCON-8

Projeto: Sistema de Frenagem Inteligente (SFI),Sistema de Velocidade Adaptativa (CVA) e Pipeline para Veículos BYD Dolphin Autonomous.
Estudante: Otávio Campos Valefuogo.
Curso: Ciência da Computação – Instituto Federal Catarinense.

1. Introdução

Este documento detalha o projeto e a implementação do processador Falcon-8, uma arquitetura RISC customizada e otimizada para sistemas embarcados de tempo real. Diferente de arquiteturas convencionais, o Falcon-8 foi projetado com um pipeline de 3 estágios para maximizar o throughput de instruções, permitindo que o Sistema de Frenagem Inteligente (SFI) reaja a obstáculos em milissegundos, superando a capacidade de resposta humana em até 300 vezes.

2. Especificação da ISA (Instruções e Formatos)

O processador Falcon-8 utiliza palavras de instrução de 16 bits, seguindo o padrão de arquitetura RISC. O decodificador da Unidade de Controle identifica a operação através do Opcode.
Abaixo estão listados os formatos suportados e a tabela completa de Opcodes mapeados no hardware:

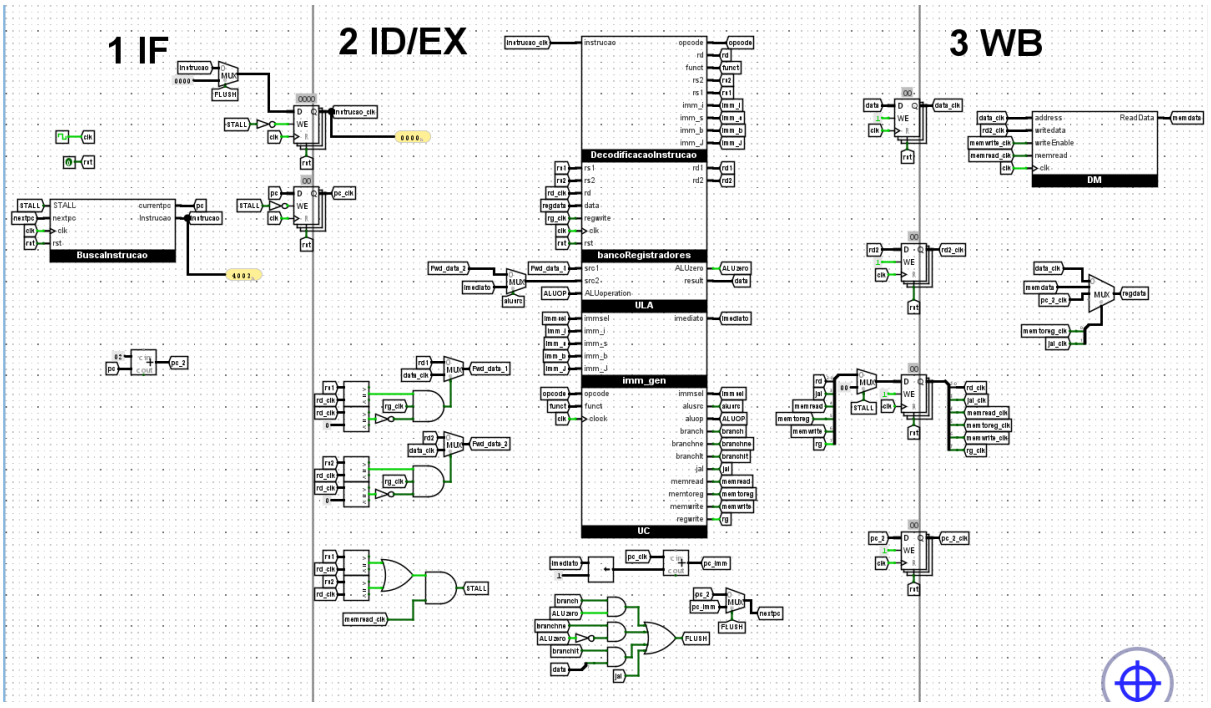
- **Tipo R (Registrador):** Operações aritméticas e lógicas entre registradores (ex: ADD, SUB).
    - **Opcode:** 0000.
    - **Detalhe de Hardware:** Neste formato, o Opcode serve apenas para avisar a Unidade de Controle que se trata de uma instrução Tipo R. O que realmente decide qual operação matemática a ULA vai executar não é o Opcode, mas sim o campo funct. A Unidade de Controle extrai esse campo funct e o converte no sinal de controle aluop, que é injetado diretamente na ULA para selecionar a operação exata.
- **Tipo I (Aritmético):** Operações matemáticas utilizando um valor constante.
    - **Opcode:** 0001 (ex: ADDI).
- **Tipo I (Load):** Carga de dados da memória RAM para o banco de registradores.
    - **Opcode:** 0010 (LW).
- **Tipo I (Shift):** Operação de deslocamento lógico de bits.
    - **Opcode:** 0111 (SLLI).

- **Tipo S (Store):** Armazenamento de dados do banco de registradores para a memória RAM.
    - **Opcode:** 0011 (SW).
- **Tipo B (Branches Condicionais):** Instruções de desvio de fluxo baseadas em comparações da ULA.
    - **Opcode:** 0100 (BEQ - Salta se igual).
    - **Opcode:** 0101 (BNE - Salta se diferente).
    - **Opcode:** 1000 (BLT - Salta se menor que).
- **Tipo J (Jump):** Saltos incondicionais absolutos.
    - **Opcode:** 0110 (JAL).

3. Arquitetura do Datapath (Pipeline de 3 Estágios)

Para alcançar o desempenho exigido pela BYD, o datapath foi dividido em duas paredes de registradores, permitindo o paralelismo em nível de instrução.

1. **Estágio 1 (Busca - IF):** Recuperação da instrução na ROM e atualização do PC.
2. **Estágio 2 (Execução/Decodificação - ID/EX):** Unidade de Controle Principal (UC), Banco de Registradores (portas de leitura), Gerador de Imediatos , Unidade Lógica e Aritmética (ULA), Somador de Endereços de Salto e os MUXes da Unidade de Detecção de Hazards/Forwarding.
3. **Estágio 3 (Gravação - WB):** Acesso à memória RAM e escrita dos resultados de volta no Banco de Registradores.

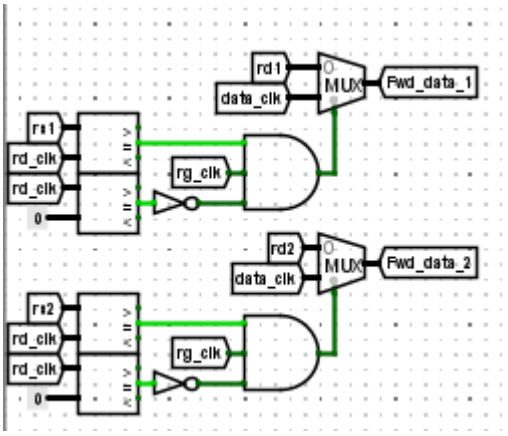


4. Tratamento de Conflitos (Hazards)

O maior desafio de um pipeline é garantir a integridade dos dados quando as instruções dependem umas das outras. O Falcon-8 implementa três soluções:

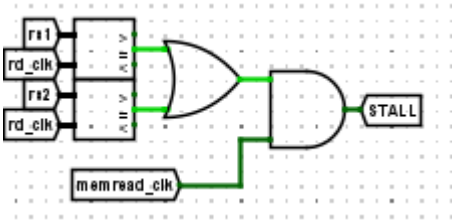
4.1. Dependências de Dados e Forwarding (Adiantamento)

Para evitar que o processador pare sempre que uma instrução precisa de um resultado que ainda não foi gravado no banco, implementamos o Forwarding. Um conjunto de MUXes antes da ULA monitora o estágio de Gravação e "adianta" o dado diretamente para a entrada da ULA se houver uma dependência de dados (ou seja, se rs1 ou rs2 forem iguais a rd_clk), o MUX atua: ele descarta o valor desatualizado que acabou de ser lido do Banco de Registradores e o substitui pelo fio data_clk (que carrega o resultado recém-calculado pela ULA no ciclo anterior). Esse dado é injetado diretamente na entrada da ULA, resolvendo o hazard instantaneamente.



4.2. Dependência de Dados Load-Use e Stall (Congelamento)

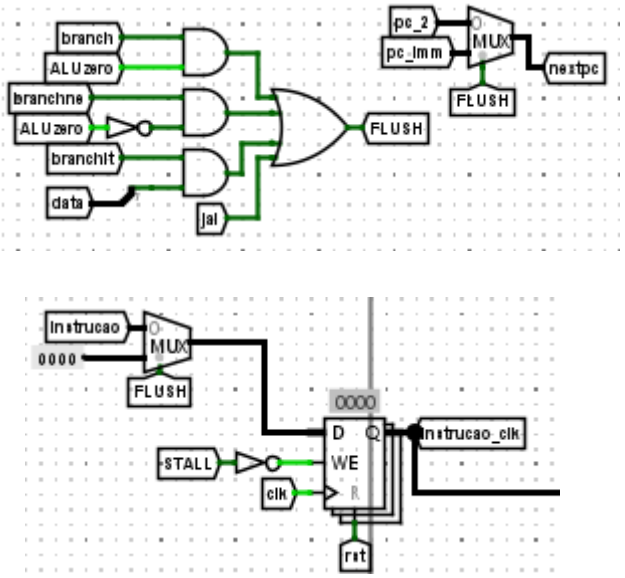
Quando uma instrução depende de um LW (leitura de memória), o dado só fica pronto no final do estágio 3. Nestes casos o Forwarding não consegue resolver assim a Unidade de Detecção de Hazards, que compara se rs1 ou rs2 forem iguais a rd_clk e (memread_clk = 1) ou seja do tipo i load, ativa o sinal de STALL, que congela o PC e a Parede 1 por um ciclo, injetando uma bolha (NOP) no controle.



4.3. Control Hazards e Hardware Flush (Descarte)

Em instruções de salto (BEQ, BNE, JAL), o processador pode buscar a instrução errada enquanto calcula o destino.

- **Solução:** Implementamos um Hardware Flush. Sempre que o salto é confirmado (combinacionalmente no Estágio 2), um sinal de FLUSH ativa um MUX na entrada da Parede 1, substituindo a instrução invasora por 0000 (Bolha).



5. Conclusão e Análise de Desempenho Matemático

A implementação do pipeline de 3 estágios no Falcon-8 resultou em um ganho exponencial de desempenho em relação a uma arquitetura monociclo. O processador monociclo é limitado pelo seu caminho crítico longo: o relógio precisa aguardar uma única instrução atravessar todos os componentes sequencialmente.

Para demonstrar essa vantagem estrutural, estimamos os seguintes tempos **fictícios** de propagação (atrasos físicos) para os componentes primários:

- **Leitura/Escrita de Memória (ROM/RAM):** 200ps
- **Decodificação e Leitura de Registradores:** 100ps
- **Atraso de Multiplexadores (MUXes):** 30ps
- **Cálculo da ULA:** 200ps
- **Setup das Paredes de Pipeline:** 20ps

5.1. O Cenário Monociclo

No processador monociclo, o tempo do relógio deve acomodar a soma de todos os atrasos da instrução mais lenta (como um LW, que utiliza todos os blocos sequencialmente).

Tempo Monociclo= ROM + RegRead + MUX_{alu} + ULA + RAM + MUX_{wb}

Tempo Monociclo= 200 + 100 + 30 + 200 + 200 + 30 = 760ps

Neste cenário, o sistema necessita de um ciclo de **760ps** por instrução.

5.2. O Cenário Pipeline (Falcon-8)

No datapath de 3 estágios, o tempo do relógio é determinado pelo estágio com o maior caminho crítico (o estágio mais lento), acrescido do tempo de setup da parede de registradores.

- **Estágio 1 (IF):** 200ps (ROM).
- **Estágio 2 (ID/EX):** Os componentes iniciais operam em paralelo (Unidade de Controle, Gerador de Imediato e Banco de Registradores). O caminho crítico é formado pela leitura do Banco, passagem pelos MUXes de Forwarding/ALUSrc e o cálculo da ULA.
  RegRead (100ps) + MUX (30ps) + ULA (200ps) = 330ps
- **Estágio 3 (MEM/WB):** 200ps (RAM) + 30ps (MUX de Write-Back) = 230ps.

O estágio mais lento é o Estágio 2 (330ps). Adicionando o setup da parede (20ps), temos o ciclo ideal do pipeline:

Tempo Pipeline= 330ps + 20ps = 350ps

5.3. O Fator de Aceleração

A relação de ganho de desempenho estrutural é calculada dividindo o ciclo do monociclo pelo ciclo do pipeline:

Ganho Desempenho= Tempo Monociclo/Tempo Pipeline

Ganho Desempenho= 760/350=2.17

5.4. Impacto Real no Sistema BYD

A matemática do hardware comprova que o Falcon-8 em pipeline opera **2.17 vezes mais rápido** que sua versão monociclo. Em um cenário real de direção autônoma (SFI), essa drástica redução no tempo de ciclo permite que o processador calcule matrizes de distância e avalie saltos condicionais na metade do tempo. Com os Hazards totalmente resolvidos em hardware por Forwarding e Flush, o sistema não sofre quedas abruptas de desempenho (CPI), garantindo a velocidade de reação extrema necessária para parar o veículo e salvar vidas.